Clean Architecture Guide for .NET Microservices

Reference Implementation: OfferCraft.Lobby.Microservice
Version: 1.0
Last Updated: January 27, 2026
Status: Production Standard

Table of Contents

---

1. Introduction

What is Clean Architecture?

Clean Architecture is a software design pattern that **separates business logic from technical implementation details**. It ensures your application is:

- **Independent of frameworks** - Not locked to EF Core, ASP.NET, etc.
- **Testable** - Business logic can be tested without UI, database, or external services
- **Independent of UI** - Can change from REST API to GraphQL without touching business logic
- **Independent of Database** - Can swap PostgreSQL for MongoDB without changing core logic
- **Independent of external agencies** - Business rules don't know about external APIs

Why Use Clean Architecture?

Short-term benefits: - Clear organization - Everyone knows where to put code - Easy to find things - Predictable structure - Faster code reviews - Standard patterns

Long-term benefits: - Easy to maintain - Changes are isolated - Easy to test - Layers can be tested independently - Easy to scale - Add features without breaking existing code - Technology independence - Swap databases, frameworks without rewriting business logic

The Core Philosophy

> **"Your business logic should not care about technical details."**

A `Lobby` entity doesn't care if it's stored in PostgreSQL, MongoDB, or a text file. It only cares about being a Lobby with business rules.

---

2. Clean Architecture Principles

The Dependency Rule

THE MOST IMPORTANT RULE:

```
Dependencies must point INWARD (toward business logic)


  API (Controllers)                          ← Outer Layer
  ↓ depends on

     Application (Services, DTOs)
     ↓ depends on

        Domain (Entities, Rules)            ← Core (Inner Layer)
        ← NEVER depends on outer

     ↑ depended upon by
     Infrastructure (Database)
```

Key Points: - Inner layers NEVER reference outer layers - Outer layers can reference inner layers - Domain is at the center - it knows NOTHING about the outside world - Infrastructure depends on Domain, NOT the other way around

---

3. Layer Structure

The Four Layers

```
YourProject.Microservice/
  src/
      YourProject.Domain/          ← Layer 1: Business Entities & Rules
      YourProject.Application/     ← Layer 2: Business Logic & Use Cases
      YourProject.Infrastructure/  ← Layer 3: Data Access & External Services
      YourProject.Api/             ← Layer 4: API Controllers & Entry Point
  tests/
       YourProject.UnitTests/
       YourProject.IntegrationTests/
```

Dependency Flow Diagram

```
API

  ↓                  ↓
Application    Infrastructure


      → Domain ←
        (Core - No dependencies)
```

---

4. The Four Layers Explained

Layer 1: Domain Layer

Purpose: Contains your **business entities and business rules**. This is the heart of your application.

What goes here: - Business entities (classes representing your business concepts) - Value objects (immutable objects with business meaning) - Domain interfaces (contracts that outer layers implement) - Domain exceptions (business rule violations) - Enums and constants

What does NOT go here: - Database code (DbContext, migrations) - API code (controllers, routes) - External service integrations - Framework dependencies (except .NET base libraries) - DTOs (those belong in Application layer)

Key Characteristics: - **Zero external dependencies** (no NuGet packages except maybe domain-specific libraries) - **Pure C# classes** representing busi-

ness concepts - **Technology agnostic** - no knowledge of databases, APIs, or frameworks

Example Structure:

```
Domain/
  Entities/
     Lobby.cs                    ← Main business entity
     LobbyType.cs                ← Lookup/reference entity
     LobbyCampaignItem.cs        ← Related entity
  Interfaces/
     IAuditable.cs               ← Marker interface for audit trail
     ISoftDeletable.cs           ← Marker interface for soft delete
     ILobbyRepository.cs         ← Repository contract (implemented by Infrastructure)
     IUnitOfWork.cs              ← Transaction contract
  Exceptions/
     LobbyNotFoundException.cs   ← Domain-specific exception
     InvalidLobbyStateException.cs
  Enums/
      LobbyStatus.cs              ← Business status enum
      CampaignOrderType.cs
```

Real Example from OfferCraft.Lobby:

```
// Domain/Entities/Lobby.cs
namespace OfferCraft.Lobby.Domain.Entities;

/// <summary>
/// Represents a lobby in the OfferCraft system.
/// This is a BUSINESS ENTITY - it knows nothing about databases or APIs.
/// </summary>
public class Lobby : IAuditable, ISoftDeletable
{
    // Business Properties
    public Guid LobbyId { get; set; }
    public Guid AccountId { get; set; }
    public string Title { get; set; } = string.Empty;
    public string? Description { get; set; }

    // Audit Properties (from IAuditable)
    public DateTime CreatedOn { get; set; }
    public Guid? CreatedBy { get; set; }
    public DateTime ModifiedOn { get; set; }
    public Guid? ModifiedBy { get; set; }

    // Soft Delete Properties (from ISoftDeletable)
    public bool? IsDeleted { get; set; }
    public DateTime? DeletedOn { get; set; }
```

```
    // Navigation Properties (business relationships)
    public virtual ICollection<LobbyCampaignItem> CampaignItems { get; set; } = new List<Lob

    // NO database attributes here!
    // NO [Table], [Column], [Key] attributes!
    // This is a pure business object!
}

// Domain/Interfaces/IAuditable.cs
namespace OfferCraft.Lobby.Domain.Interfaces;

/// <summary>
/// Marker interface for entities that need automatic audit trail.
/// This is a DOMAIN CONCEPT - audit trail is a business requirement.
/// </summary>
public interface IAuditable
{
    DateTime CreatedOn { get; set; }
    Guid? CreatedBy { get; set; }
    DateTime ModifiedOn { get; set; }
    Guid? ModifiedBy { get; set; }
}

// Domain/Interfaces/ILobbyRepository.cs
namespace OfferCraft.Lobby.Domain.Interfaces;

/// <summary>
/// Repository contract - Domain defines WHAT it needs, Infrastructure defines HOW.
/// This interface lives in Domain but is IMPLEMENTED in Infrastructure.
/// </summary>
public interface ILobbyRepository
{
    Task<Lobby> GetByIdAsync(Guid id, CancellationToken cancellationToken = default);
    Task<IEnumerable<Lobby>> GetActiveAsync(CancellationToken cancellationToken = default);
    Task AddAsync(Lobby lobby, CancellationToken cancellationToken = default);
    void Update(Lobby lobby);
    void Delete(Lobby lobby);
}
```

Why This Layer Exists: - Contains your **competitive advantage** - your unique business logic - Can be moved to a different framework/technology without changes - Can be tested without any infrastructure (no database, no API)

What Happens if You Violate This Layer: - Adding `using Microsoft.EntityFrameworkCore;` → Your business logic is now tied to EF Core - Adding `[Table("lobbies")]` attribute → Your entity knows about database structure - Adding `ILogger` dependency → Your entity depends on

a framework

Layer 2: Application Layer

Purpose: Contains **business logic and use cases**. Orchestrates Domain entities to fulfill business scenarios.

What goes here: - Service interfaces (`ILobbyService`) - Service implementations (`LobbyService`) - DTOs (Data Transfer Objects - `CreateLobbyRequest`, `LobbyDto`) - Validators (`CreateLobbyRequestValidator`) - AutoMapper profiles (`LobbyMappingProfile`) - Business logic that orchestrates domain entities

What does NOT go here: - Database code (DbContext, SQL queries) - HTTP/API concerns (attributes like [HttpPost], route definitions) - Entity Framework configurations - Database connection strings

Dependencies Allowed: - Reference Domain project - FluentValidation - AutoMapper - NO Entity Framework Core - NO ASP.NET Core (only abstractions like `IHttpContextAccessor` if needed)

Example Structure:

```
Application/
  DTOs/
      CreateLobbyRequest.cs       ← Input DTO for creating lobby
      UpdateLobbyRequest.cs       ← Input DTO for updating lobby
      LobbyDto.cs                 ← Output DTO for lobby details
      LobbyListItemDto.cs         ← Output DTO for lobby list
  Interfaces/
      ILobbyService.cs            ← Service contract
      ICurrentUserService.cs      ← User context abstraction
  Services/
      LobbyService.cs             ← Service implementation (business logic)
  Validators/
      CreateLobbyRequestValidator.cs  ← FluentValidation rules
      UpdateLobbyRequestValidator.cs
  Mappings/
       LobbyMappingProfile.cs      ← AutoMapper configuration
```

Real Example from OfferCraft.Lobby:

```
// Application/DTOs/CreateLobbyRequest.cs
namespace OfferCraft.Lobby.Application.DTOs;

/// <summary>
/// DTO for creating a new lobby.
/// DTOs live in Application layer - they represent API contracts, not business entities.
```

```
/// </summary>
public class CreateLobbyRequest
{
    public Guid AccountId { get; set; }
    public string Title { get; set; } = string.Empty;
    public string? Description { get; set; }
    public int LobbyTypeId { get; set; } = 1;
    public int LobbyModeId { get; set; } = 1;
    public bool RequiresLogin { get; set; }
    public string? Locale { get; set; }

    // This is NOT a domain entity - it's a data transfer object
    // It represents what the API consumer sends, not the business model
}
```

```
// Application/Interfaces/ILobbyService.cs
namespace OfferCraft.Lobby.Application.Interfaces;

/// <summary>
/// Service contract defining business operations for lobbies.
/// Notice: NO userId parameters - audit trail is handled automatically by Infrastructure.
/// </summary>
public interface ILobbyService
{
    Task<LobbyDto> CreateAsync(CreateLobbyRequest request, CancellationToken cancellationTok
    Task<LobbyDto> UpdateAsync(Guid id, UpdateLobbyRequest request, CancellationToken cancel
    Task<LobbyDto> GetByIdAsync(Guid id, CancellationToken cancellationToken = default);
    Task<IEnumerable<LobbyListItemDto>> GetActiveAsync(CancellationToken cancellationToken =
    Task DeleteAsync(Guid id, CancellationToken cancellationToken = default);
    Task ArchiveAsync(Guid id, CancellationToken cancellationToken = default);
    Task<LobbyDto> CloneAsync(Guid id, CancellationToken cancellationToken = default);
}
```

```
// Application/Services/LobbyService.cs
namespace OfferCraft.Lobby.Application.Services;

/// <summary>
/// Business logic implementation.
/// This orchestrates domain entities to fulfill business use cases.
/// </summary>
public class LobbyService : ILobbyService
{
    private readonly IUnitOfWork _unitOfWork;
    private readonly IMapper _mapper;

    public LobbyService(IUnitOfWork unitOfWork, IMapper mapper)
    {
```

```
        _unitOfWork = unitOfWork;
        _mapper = mapper;
    }

    public async Task<LobbyDto> CreateAsync(CreateLobbyRequest request, CancellationToken ca
    {
        // 1. Map DTO to Domain Entity
        var lobby = _mapper.Map<Lobby>(request);

        // 2. Set business defaults
        lobby.LobbyId = Guid.NewGuid();
        lobby.IsArchived = false;
        lobby.IsDisabled = false;

        // 3. NO audit code here! Infrastructure handles this automatically
        // 4. Save using repository
        await _unitOfWork.Lobbies.AddAsync(lobby, cancellationToken);
        await _unitOfWork.SaveChangesAsync(cancellationToken);

        // 5. Map Domain Entity to DTO for response
        return _mapper.Map<LobbyDto>(lobby);
    }

    // Notice: This is BUSINESS LOGIC, not database logic
    // We work with domain entities, not database tables
}

// Application/Validators/CreateLobbyRequestValidator.cs
namespace OfferCraft.Lobby.Application.Validators;

/// <summary>
/// Validation rules for creating a lobby.
/// Business validation logic lives in Application layer.
/// </summary>
public class CreateLobbyRequestValidator : AbstractValidator<CreateLobbyRequest>
{
    public CreateLobbyRequestValidator()
    {
        RuleFor(x => x.AccountId)
            .NotEmpty().WithMessage("Account ID is required");

        RuleFor(x => x.Title)
            .NotEmpty().WithMessage("Title is required")
            .MaximumLength(255).WithMessage("Title cannot exceed 255 characters");

        RuleFor(x => x.LobbyTypeId)
```

```
            .GreaterThan(0).WithMessage("Valid lobby type is required");
    }
}
```

Why This Layer Exists: - Orchestrates domain entities to fulfill use cases - Provides a stable interface for API layer - Contains business logic that spans multiple entities - Maps between DTOs (external representation) and Entities (internal representation)

What Happens if You Violate This Layer: - Writing SQL in services → Business logic tied to specific database - Using `DbContext` directly → Cannot test without database - Returning entities instead of DTOs → Internal structure exposed to API

Layer 3: Infrastructure Layer

Purpose: Contains **implementations of technical concerns** - how you persist data, call external APIs, send emails, etc.

What goes here: - DbContext and EF Core configurations - Repository implementations - Database migrations - External API clients - File system access - Email/SMS services - Caching implementations - Current user service (extracts user from HTTP context)

What does NOT go here: - Business logic (that's in Application) - Domain entities (those are in Domain) - Controllers (those are in API)

Dependencies Allowed: - Reference Domain project (implements interfaces defined there) - Reference Application project (for abstractions like `ICurrentUserService`) - Entity Framework Core - Npgsql (PostgreSQL provider) - Any external SDKs/libraries

Example Structure:

```
Infrastructure/
  Data/
     LobbyDbContext.cs           ← EF Core DbContext with configurations
     Migrations/                 ← EF Core migrations
         20260119_InitialCreate.cs
         ...
  Repositories/
     LobbyRepository.cs          ← Implements ILobbyRepository from Domain
     UnitOfWork.cs               ← Implements IUnitOfWork from Domain
  Services/
      CurrentUserService.cs       ← Implements ICurrentUserService from Application
```

Real Example from OfferCraft.Lobby:

```
// Infrastructure/Data/LobbyDbContext.cs
using Microsoft.EntityFrameworkCore;
using OfferCraft.Lobby.Domain.Entities;
using OfferCraft.Lobby.Domain.Interfaces;
using OfferCraft.Lobby.Application.Interfaces;

namespace OfferCraft.Lobby.Infrastructure.Data;

/// <summary>
/// DbContext with automatic audit trail.
/// This is INFRASTRUCTURE - it knows about PostgreSQL, EF Core, and database columns.
/// </summary>
public class LobbyDbContext : DbContext
{
    private readonly ICurrentUserService _currentUserService;

    public LobbyDbContext(
        DbContextOptions<LobbyDbContext> options,
        ICurrentUserService currentUserService) : base(options)
    {
        _currentUserService = currentUserService;
    }

    // DbSets - map domain entities to database tables
    public DbSet<Lobby> Lobbies { get; set; }
    public DbSet<LobbyType> LobbyTypes { get; set; }
    public DbSet<LobbyCampaignItem> LobbyCampaignItems { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // Infrastructure decides HOW to map domain entities to database
        modelBuilder.Entity<Lobby>(entity =>
        {
            entity.ToTable("lobbies", "dbo");  // Table name
            entity.HasKey(e => e.LobbyId);

            // Map C# property names to database column names
            entity.Property(e => e.LobbyId).HasColumnName("lobby_id");
            entity.Property(e => e.AccountId).HasColumnName("account_id");
            entity.Property(e => e.Title).HasColumnName("title").HasMaxLength(255);
            entity.Property(e => e.CreatedOn).HasColumnName("created_on");
            entity.Property(e => e.CreatedBy).HasColumnName("created_by");

            // Relationships
            entity.HasMany(e => e.CampaignItems)
                  .WithOne()
```

```
                .HasForeignKey(ci => ci.LobbyId);
        });
    }

    /// <summary>
    /// Automatic audit trail - Infrastructure handles cross-cutting concerns.
    /// This is the MAGIC that eliminates manual audit code from services.
    /// </summary>
    public override async Task<int> SaveChangesAsync(CancellationToken cancellationToken = d
    {
        var userId = _currentUserService.GetUserId();
        var now = DateTime.UtcNow;

        // Find all entities that implement IAuditable
        foreach (var entry in ChangeTracker.Entries<IAuditable>())
        {
            switch (entry.State)
            {
                case EntityState.Added:
                    // New entity - set all audit fields
                    entry.Entity.CreatedOn = now;
                    entry.Entity.CreatedBy = userId;
                    entry.Entity.ModifiedOn = now;
                    entry.Entity.ModifiedBy = userId;
                    break;

                case EntityState.Modified:
                    // Updated entity - only update modified fields
                    entry.Entity.ModifiedOn = now;
                    entry.Entity.ModifiedBy = userId;
                    // Prevent EF from overwriting CreatedOn and CreatedBy
                    entry.Property(e => e.CreatedOn).IsModified = false;
                    entry.Property(e => e.CreatedBy).IsModified = false;
                    break;
            }
        }

        // Handle soft deletes
        foreach (var entry in ChangeTracker.Entries<ISoftDeletable>())
        {
            if (entry.State == EntityState.Deleted)
            {
                // Convert hard delete to soft delete
                entry.State = EntityState.Modified;
                entry.Entity.IsDeleted = true;
                entry.Entity.DeletedOn = now;
```

```
                if (entry.Entity is IAuditable auditable)
                {
                    auditable.ModifiedOn = now;
                    auditable.ModifiedBy = userId;
                }
            }
        }

        return await base.SaveChangesAsync(cancellationToken);
    }
}

// Infrastructure/Repositories/LobbyRepository.cs
using Microsoft.EntityFrameworkCore;
using OfferCraft.Lobby.Domain.Entities;
using OfferCraft.Lobby.Domain.Interfaces;
using OfferCraft.Lobby.Infrastructure.Data;

namespace OfferCraft.Lobby.Infrastructure.Repositories;

/// <summary>
/// Repository implementation - Infrastructure provides the HOW.
/// Implements interface defined in Domain layer.
/// </summary>
public class LobbyRepository : ILobbyRepository
{
    private readonly LobbyDbContext _context;

    public LobbyRepository(LobbyDbContext context)
    {
        _context = context;
    }

    public async Task<Lobby> GetByIdAsync(Guid id, CancellationToken cancellationToken = def
    {
        // Infrastructure detail: Using EF Core to query PostgreSQL
        return await _context.Lobbies
            .Include(l => l.CampaignItems)
            .Include(l => l.Tags)
            .Include(l => l.MediaResources)
            .FirstOrDefaultAsync(l => l.LobbyId == id, cancellationToken);
    }

    public async Task<IEnumerable<Lobby>> GetActiveAsync(CancellationToken cancellationToken
    {
```

```
        // Infrastructure detail: Filtering using LINQ
        return await _context.Lobbies
            .Where(l => l.IsArchived == false && l.IsDeleted != true)
            .ToListAsync(cancellationToken);
    }

    public async Task AddAsync(Lobby lobby, CancellationToken cancellationToken = default)
    {
        await _context.Lobbies.AddAsync(lobby, cancellationToken);
    }

    public void Update(Lobby lobby)
    {
        _context.Lobbies.Update(lobby);
    }

    public void Delete(Lobby lobby)
    {
        // This will trigger soft delete in SaveChangesAsync
        _context.Lobbies.Remove(lobby);
    }
}

// Infrastructure/Services/CurrentUserService.cs
using Microsoft.AspNetCore.Http;
using OfferCraft.Lobby.Application.Interfaces;

namespace OfferCraft.Lobby.Infrastructure.Services;

/// <summary>
/// Extracts current user ID from HTTP context.
/// This is INFRASTRUCTURE - it knows about HTTP, JWT tokens, etc.
/// </summary>
public class CurrentUserService : ICurrentUserService
{
    private readonly IHttpContextAccessor _httpContextAccessor;

    public CurrentUserService(IHttpContextAccessor httpContextAccessor)
    {
        _httpContextAccessor = httpContextAccessor;
    }

    public Guid GetUserId()
    {
        // TODO: In production, extract from JWT token
        // var userIdClaim = _httpContextAccessor.HttpContext?.User.FindFirst("sub")?.Value
```

```
        // return Guid.Parse(userIdClaim ?? throw new UnauthorizedAccessException());

        // For now, return placeholder
        return Guid.Parse("00000000-0000-0000-0000-000000000001");
    }
}
```

Why This Layer Exists: - Isolates technical implementation details - Implements contracts defined by Domain - Can be swapped (e.g., PostgreSQL → MongoDB) without affecting business logic - Handles cross-cutting concerns (audit trail, logging, caching)

What Happens if You Violate This Layer: - Putting business logic in repositories → Logic duplicated across data access code - DbContext used directly in Application → Cannot swap database providers - Infrastructure dictating domain structure → Business logic tied to database schema

---

Layer 4: API Layer

Purpose: The **entry point** for your application. Handles HTTP requests, routing, authentication, and response formatting.

What goes here: - Controllers (API endpoints) - Program.cs (application startup and DI configuration) - Middleware - API filters and attributes - Swagger/OpenAPI configuration - appsettings.json (configuration files)

What does NOT go here: - Business logic (that's in Application) - Database code (that's in Infrastructure) - Domain entities (use DTOs instead)

Dependencies Allowed: - Reference Application project - Reference Infrastructure project (for DI registration only) - ASP.NET Core - Swagger - Authentication libraries

Example Structure:

```
Api/
  Controllers/
      LobbiesController.cs        ← REST API endpoints
  Program.cs                      ← App startup, DI configuration
  appsettings.json                ← Configuration
  appsettings.Development.json    ← Dev-specific config
  Properties/
      launchSettings.json
```

Real Example from OfferCraft.Lobby:

```
// Api/Controllers/LobbiesController.cs
using Microsoft.AspNetCore.Mvc;
using OfferCraft.Lobby.Application.DTOs;
```

```
using OfferCraft.Lobby.Application.Interfaces;

namespace OfferCraft.Lobby.Api.Controllers;

/// <summary>
/// REST API controller for lobby operations.
/// This is the API LAYER - it handles HTTP concerns.
/// Notice: NO business logic here - only calls to ILobbyService.
/// </summary>
[ApiController]
[Route("api/lobbies")]
[Produces("application/json")]
public class LobbiesController : ControllerBase
{
    private readonly ILobbyService _lobbyService;
    private readonly ILogger<LobbiesController> _logger;

    public LobbiesController(ILobbyService lobbyService, ILogger<LobbiesController> logger)
    {
        _lobbyService = lobbyService;
        _logger = logger;
    }

    /// <summary>
    /// Get all active lobbies
    /// </summary>
    [HttpGet]
    [ProducesResponseType(typeof(IEnumerable<LobbyListItemDto>), StatusCodes.Status200OK)]
    public async Task<IActionResult> GetActive(CancellationToken cancellationToken)
    {
        try
        {
            // Controller only coordinates - delegates to service
            var lobbies = await _lobbyService.GetActiveAsync(cancellationToken);
            return Ok(lobbies);  // HTTP 200 with data
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, "Error retrieving active lobbies");
            return StatusCode(500, "An error occurred while retrieving lobbies");
        }
    }

    /// <summary>
    /// Create a new lobby
    /// </summary>
```

```
[HttpPost]
[ProducesResponseType(typeof(LobbyDto), StatusCodes.Status201Created)]
[ProducesResponseType(StatusCodes.Status400BadRequest)]
public async Task<IActionResult> Post([FromBody] CreateLobbyRequest request, Cancellatio
{
    try
    {
        if (!ModelState.IsValid)
            return BadRequest(ModelState);  // HTTP 400 for validation errors

        // Delegate to service - NO business logic here
        var lobby = await _lobbyService.CreateAsync(request, cancellationToken);

        // Return HTTP 201 with location header
        return CreatedAtAction(nameof(Get), new { id = lobby.LobbyId }, lobby);
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Error creating lobby");
        return StatusCode(500, "An error occurred while creating the lobby");
    }
}

/// <summary>
/// Update an existing lobby
/// </summary>
[HttpPut("{id:guid}")]
[ProducesResponseType(typeof(LobbyDto), StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
public async Task<IActionResult> Put(Guid id, [FromBody] UpdateLobbyRequest request, Car
{
    try
    {
        if (!ModelState.IsValid)
            return BadRequest(ModelState);

        var lobby = await _lobbyService.UpdateAsync(id, request, cancellationToken);
        return Ok(lobby);
    }
    catch (KeyNotFoundException)
    {
        return NotFound($"Lobby with ID {id} not found");
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Error updating lobby {LobbyId}", id);
```

```
            return StatusCode(500, "An error occurred while updating the lobby");
        }
    }

    // Notice: Controller is THIN - no business logic, just HTTP coordination
}

// Api/Program.cs
using OfferCraft.Lobby.Application.Interfaces;
using OfferCraft.Lobby.Application.Services;
using OfferCraft.Lobby.Application.Mappings;
using OfferCraft.Lobby.Application.Validators;
using OfferCraft.Lobby.Domain.Interfaces;
using OfferCraft.Lobby.Infrastructure.Data;
using OfferCraft.Lobby.Infrastructure.Repositories;
using OfferCraft.Lobby.Infrastructure.Services;
using Microsoft.EntityFrameworkCore;
using FluentValidation.AspNetCore;
using FluentValidation;

var builder = WebApplication.CreateBuilder(args);

// ========== DATABASE CONFIGURATION ==========
builder.Services.AddDbContext<LobbyDbContext>(options =>
    options.UseNpgsql(builder.Configuration.GetConnectionString("DefaultConnection")));

// ========== DEPENDENCY INJECTION ==========
// Infrastructure services
builder.Services.AddHttpContextAccessor();
builder.Services.AddScoped<ICurrentUserService, CurrentUserService>();

// Repositories (Infrastructure implements Domain interfaces)
builder.Services.AddScoped<ILobbyRepository, LobbyRepository>();
builder.Services.AddScoped<IUnitOfWork, UnitOfWork>();

// Application services
builder.Services.AddScoped<ILobbyService, LobbyService>();

// ========== AUTOMAPPER ==========
builder.Services.AddAutoMapper(typeof(LobbyMappingProfile));

// ========== FLUENT VALIDATION ==========
builder.Services.AddValidatorsFromAssemblyContaining<CreateLobbyRequestValidator>();
builder.Services.AddFluentValidationAutoValidation();

// ========== API CONFIGURATION ==========
```

```
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

// ========== MIDDLEWARE PIPELINE ==========
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

Why This Layer Exists: - Entry point for external requests - Handles HTTP-specific concerns (routing, status codes, headers) - Configures dependency injection - Thin layer that delegates to Application services

What Happens if You Violate This Layer: - Business logic in controllers → Logic duplicated across endpoints - Direct DbContext usage → Cannot swap database without changing controllers - Returning entities instead of DTOs → Internal structure exposed

---

5. Dependency Rules

The Sacred Rules

Rule 1: Dependencies Point Inward

```
 CORRECT:
API → Application → Domain ← Infrastructure

 WRONG:
Domain → Infrastructure (NEVER!)
Application → API (NEVER!)
```

Rule 2: Inner Layers Know Nothing About Outer Layers

```
//  CORRECT - Domain has no dependencies
namespace OfferCraft.Lobby.Domain.Entities;

public class Lobby  // Pure C# class
```

```
{
    public Guid LobbyId { get; set; }
    // No using statements for EF Core, ASP.NET, etc.
}

//  WRONG - Domain depends on Infrastructure
namespace OfferCraft.Lobby.Domain.Entities;

using Microsoft.EntityFrameworkCore;  //  NO!

[Table("lobbies")]  //  NO! Domain shouldn't know about database tables
public class Lobby
{
    [Key]  //  NO! Domain shouldn't have EF attributes
    public Guid LobbyId { get; set; }
}
```

Rule 3: Use Interfaces for Inversion of Control

```
// Domain defines WHAT it needs (interface)
// Domain/Interfaces/ILobbyRepository.cs
namespace OfferCraft.Lobby.Domain.Interfaces;

public interface ILobbyRepository
{
    Task<Lobby> GetByIdAsync(Guid id);
}

// Infrastructure implements HOW (concrete class)
// Infrastructure/Repositories/LobbyRepository.cs
namespace OfferCraft.Lobby.Infrastructure.Repositories;

public class LobbyRepository : ILobbyRepository  // Implements Domain interface
{
    private readonly LobbyDbContext _context;  // Infrastructure detail

    public async Task<Lobby> GetByIdAsync(Guid id)
    {
        return await _context.Lobbies.FindAsync(id);  // EF Core usage
    }
}
```

Project Reference Rules

Allowed References:

```
API → Application
```

```
API → Infrastructure (only for DI registration in Program.cs)
Application → Domain
Infrastructure → Domain
Infrastructure → Application (only for abstractions)

Domain → NOTHING (zero external dependencies)
Application → Infrastructure
Application → API
Infrastructure → API
```

How to Check Your Dependencies

Domain.csproj should look like this:

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net9.0</TargetFramework>
  </PropertyGroup>

  <ItemGroup>
    <!-- NO PROJECT REFERENCES! -->
    <!-- NO NUGET PACKAGES (except maybe domain-specific libraries)! -->
  </ItemGroup>
</Project>
```

Application.csproj should look like this:

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net9.0</TargetFramework>
  </PropertyGroup>

  <ItemGroup>
    <!-- ONLY Domain reference -->
    <ProjectReference Include="..\OfferCraft.Lobby.Domain\OfferCraft.Lobby.Domain.csproj" />
  </ItemGroup>

  <ItemGroup>
    <!-- Application-specific packages OK -->
    <PackageReference Include="AutoMapper" Version="13.0.1" />
    <PackageReference Include="FluentValidation" Version="11.3.0" />
  </ItemGroup>
</Project>
```

---

6. Project Structure Template

Complete Folder Structure

```
YourProject.Microservice/

  src/

      YourProject.Domain/
          Entities/
              YourMainEntity.cs
              YourRelatedEntity.cs
              YourLookupEntity.cs
          Interfaces/
              IAuditable.cs
              ISoftDeletable.cs
              IYourMainRepository.cs
              IUnitOfWork.cs
          Exceptions/
              YourEntityNotFoundException.cs
              InvalidYourEntityStateException.cs
          Enums/
              YourEntityStatus.cs
          YourProject.Domain.csproj

      YourProject.Application/
          DTOs/
              CreateYourEntityRequest.cs
              UpdateYourEntityRequest.cs
              YourEntityDto.cs
              YourEntityListItemDto.cs
          Interfaces/
              IYourEntityService.cs
              ICurrentUserService.cs
          Services/
              YourEntityService.cs
          Validators/
              CreateYourEntityRequestValidator.cs
              UpdateYourEntityRequestValidator.cs
          Mappings/
              YourEntityMappingProfile.cs
          YourProject.Application.csproj

      YourProject.Infrastructure/
          Data/
              YourProjectDbContext.cs
```

```
            Migrations/
                (EF Core migrations)
        Repositories/
            YourMainRepository.cs
            UnitOfWork.cs
        Services/
            CurrentUserService.cs
        YourProject.Infrastructure.csproj

    YourProject.Api/
        Controllers/
            YourEntitiesController.cs
        Program.cs
        appsettings.json
        appsettings.Development.json
        YourProject.Api.csproj

tests/
    YourProject.UnitTests/
        DTOs/
        Entities/
        Services/
        Repositories/
        Validators/

    YourProject.IntegrationTests/
        Controllers/

YourProject.sln
README.md
CLEAN-ARCHITECTURE-GUIDE.md
Dockerfile
docker-compose.yml
.gitignore
```

---

7. What Goes Where - Complete Guide

Quick Reference Table

Item	Layer	Why
Business entities (Customer, Order, Product)	Domain	They represent business concepts

Item	Layer	Why
Interfaces (ICustomerRepository)	Domain	Define contracts that Infrastructure implements
Enums (OrderStatus, PaymentMethod)	Domain	Business concepts
Business exceptions (OrderNotFoundException)	Domain	Business rule violations
DTOs (CreateOrderRequest, OrderDto)	Application	API contracts, not business models
Service interfaces (IOrderService)	Application	Define business operations
Service implementations (OrderService)	Application	Orchestrate domain entities
Validators (CreateOrderValidator)	Application	Validate input from API
AutoMapper profiles	Application	Map between DTOs and entities
DbContext	Infrastructure	Database access implementation
Repository implementations	Infrastructure	Data access implementation
Migrations	Infrastructure	Database schema changes
External API clients	Infrastructure	Third-party integrations
CurrentUserService	Infrastructure	HTTP context access
Controllers	API	HTTP endpoints
Program.cs	API	Startup and DI
Middleware	API	HTTP pipeline
appsettings.json	API	Configuration

Decision Tree: Where Does This Code Go?

```
START HERE

  Is it a business concept (Customer, Order, Product)?
    YES → Domain/Entities/

  Is it a contract for data access (ICustomerRepository)?
    YES → Domain/Interfaces/
```

```
Is it data coming FROM or going TO the API (CreateCustomerRequest)?
  YES → Application/DTOs/

Does it orchestrate multiple entities (create order + inventory + payment)?
  YES → Application/Services/

Does it validate API input?
  YES → Application/Validators/

Does it access a database?
  YES → Infrastructure/Repositories/ or Infrastructure/Data/

Does it call an external API?
  YES → Infrastructure/Services/ (e.g., PaymentGatewayService)

Does it handle HTTP requests?
  YES → API/Controllers/

Is it configuration or startup code?
   YES → API/Program.cs or API/appsettings.json
```

Common Scenarios

Scenario 1: Adding a New Feature (e.g., "Campaigns") Step 1: Domain Layer

```
// Domain/Entities/Campaign.cs
public class Campaign : IAuditable, ISoftDeletable
{
    public Guid CampaignId { get; set; }
    public string Name { get; set; }
    public DateTime StartDate { get; set; }
    public DateTime EndDate { get; set; }
    // Business properties only
}

// Domain/Interfaces/ICampaignRepository.cs
public interface ICampaignRepository
{
    Task<Campaign> GetByIdAsync(Guid id);
    Task<IEnumerable<Campaign>> GetActiveAsync();
}
```

Step 2: Application Layer

```
// Application/DTOs/CreateCampaignRequest.cs
```

```
public class CreateCampaignRequest
{
    public string Name { get; set; }
    public DateTime StartDate { get; set; }
    public DateTime EndDate { get; set; }
}

// Application/DTOs/CampaignDto.cs
public class CampaignDto
{
    public Guid CampaignId { get; set; }
    public string Name { get; set; }
    public DateTime StartDate { get; set; }
    public DateTime EndDate { get; set; }
}

// Application/Interfaces/ICampaignService.cs
public interface ICampaignService
{
    Task<CampaignDto> CreateAsync(CreateCampaignRequest request);
    Task<CampaignDto> GetByIdAsync(Guid id);
}

// Application/Services/CampaignService.cs
public class CampaignService : ICampaignService
{
    private readonly IUnitOfWork _unitOfWork;
    private readonly IMapper _mapper;

    public async Task<CampaignDto> CreateAsync(CreateCampaignRequest request)
    {
        var campaign = _mapper.Map<Campaign>(request);
        campaign.CampaignId = Guid.NewGuid();

        await _unitOfWork.Campaigns.AddAsync(campaign);
        await _unitOfWork.SaveChangesAsync();

        return _mapper.Map<CampaignDto>(campaign);
    }
}

// Application/Validators/CreateCampaignRequestValidator.cs
public class CreateCampaignRequestValidator : AbstractValidator<CreateCampaignRequest>
{
    public CreateCampaignRequestValidator()
    {
```

```
        RuleFor(x => x.Name).NotEmpty().MaximumLength(255);
        RuleFor(x => x.StartDate).LessThan(x => x.EndDate);
    }
}
```

Step 3: Infrastructure Layer

```
// Infrastructure/Data/YourDbContext.cs (add DbSet)
public DbSet<Campaign> Campaigns { get; set; }

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Campaign>(entity =>
    {
        entity.ToTable("campaigns", "dbo");
        entity.HasKey(e => e.CampaignId);
        entity.Property(e => e.Name).HasColumnName("name").HasMaxLength(255);
    });
}

// Infrastructure/Repositories/CampaignRepository.cs
public class CampaignRepository : ICampaignRepository
{
    private readonly YourDbContext _context;

    public async Task<Campaign> GetByIdAsync(Guid id)
    {
        return await _context.Campaigns.FindAsync(id);
    }
}
```

Step 4: API Layer

```
// Api/Controllers/CampaignsController.cs
[ApiController]
[Route("api/campaigns")]
public class CampaignsController : ControllerBase
{
    private readonly ICampaignService _campaignService;

    [HttpPost]
    public async Task<IActionResult> Post([FromBody] CreateCampaignRequest request)
    {
        var campaign = await _campaignService.CreateAsync(request);
        return CreatedAtAction(nameof(Get), new { id = campaign.CampaignId }, campaign);
    }

    [HttpGet("{id:guid}")]
```

```
    public async Task<IActionResult> Get(Guid id)
    {
        var campaign = await _campaignService.GetByIdAsync(id);
        return Ok(campaign);
    }
}

// Api/Program.cs (register dependencies)
builder.Services.AddScoped<ICampaignRepository, CampaignRepository>();
builder.Services.AddScoped<ICampaignService, CampaignService>();
```

Scenario 2: Adding External Integration (e.g., Email Service) Domain Layer:

```
// Domain/Interfaces/IEmailService.cs
public interface IEmailService
{
    Task SendAsync(string to, string subject, string body);
}
```

Application Layer:

```
// Application/Services/CustomerService.cs
public class CustomerService
{
    private readonly IEmailService _emailService;  // Use abstraction

    public async Task RegisterCustomerAsync(CreateCustomerRequest request)
    {
        var customer = _mapper.Map<Customer>(request);
        await _unitOfWork.Customers.AddAsync(customer);
        await _unitOfWork.SaveChangesAsync();

        // Send welcome email
        await _emailService.SendAsync(
            customer.Email,
            "Welcome!",
            $"Hello {customer.Name}, welcome to our platform!");
    }
}
```

Infrastructure Layer:

```
// Infrastructure/Services/SendGridEmailService.cs
public class SendGridEmailService : IEmailService
{
    private readonly ISendGridClient _client;
```

```
    public async Task SendAsync(string to, string subject, string body)
    {
        var msg = new SendGridMessage
        {
            From = new EmailAddress("noreply@example.com"),
            Subject = subject,
            PlainTextContent = body
        };
        msg.AddTo(new EmailAddress(to));

        await _client.SendEmailAsync(msg);
    }
}
```

API Layer:

```
// Api/Program.cs
builder.Services.AddScoped<IEmailService, SendGridEmailService>();
```

---

8. Common Mistakes and How to Avoid Them

Mistake 1: Business Logic in Controllers

WRONG:

```
// Api/Controllers/OrdersController.cs
[HttpPost]
public async Task<IActionResult> CreateOrder([FromBody] CreateOrderRequest request)
{
    //  Business logic in controller!
    var order = new Order
    {
        OrderId = Guid.NewGuid(),
        CustomerId = request.CustomerId,
        Total = request.Items.Sum(i => i.Price * i.Quantity),  //  Calculation
        Status = OrderStatus.Pending,
        CreatedOn = DateTime.UtcNow  //  Manual audit
    };

    await _context.Orders.AddAsync(order);  //  Direct DbContext usage
    await _context.SaveChangesAsync();

    return Ok(order);  //  Returning entity instead of DTO
}
```

CORRECT:

```
// Api/Controllers/OrdersController.cs
[HttpPost]
public async Task<IActionResult> CreateOrder([FromBody] CreateOrderRequest request)
{
    //  Delegate to service
    var order = await _orderService.CreateAsync(request);
    return CreatedAtAction(nameof(GetOrder), new { id = order.OrderId }, order);
}

// Application/Services/OrderService.cs
public async Task<OrderDto> CreateAsync(CreateOrderRequest request)
{
    //  Business logic in service
    var order = _mapper.Map<Order>(request);
    order.OrderId = Guid.NewGuid();
    order.CalculateTotal();  // Business logic method

    await _unitOfWork.Orders.AddAsync(order);
    await _unitOfWork.SaveChangesAsync();  //  Audit handled automatically

    return _mapper.Map<OrderDto>(order);
}
```

Mistake 2: Domain Entities with Database Attributes

WRONG:

```
// Domain/Entities/Customer.cs
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

[Table("customers")]  //  Domain knows about database
public class Customer
{
    [Key]  //  EF Core attribute
    [Column("customer_id")]  //  Column mapping
    public Guid CustomerId { get; set; }

    [Required]  //  Validation attribute (use FluentValidation instead)
    [MaxLength(255)]  //  Database constraint
    public string Name { get; set; }
}
```

CORRECT:

```
// Domain/Entities/Customer.cs
public class Customer : IAuditable  //  Pure business entity
```

```
{
    public Guid CustomerId { get; set; }
    public string Name { get; set; }
    public string Email { get; set; }
    // No attributes!
}

// Infrastructure/Data/YourDbContext.cs
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    //  Infrastructure handles database mapping
    modelBuilder.Entity<Customer>(entity =>
    {
        entity.ToTable("customers", "dbo");
        entity.HasKey(e => e.CustomerId);
        entity.Property(e => e.CustomerId).HasColumnName("customer_id");
        entity.Property(e => e.Name).HasColumnName("name").HasMaxLength(255).IsRequired();
    });
}

// Application/Validators/CreateCustomerRequestValidator.cs
public class CreateCustomerRequestValidator : AbstractValidator<CreateCustomerRequest>
{
    public CreateCustomerRequestValidator()
    {
        //  Application handles validation
        RuleFor(x => x.Name).NotEmpty().MaximumLength(255);
        RuleFor(x => x.Email).EmailAddress();
    }
}
```

Mistake 3: Using DbContext Directly in Application Layer

WRONG:

```
// Application/Services/CustomerService.cs
using Microsoft.EntityFrameworkCore;  //  Application depends on EF Core

public class CustomerService
{
    private readonly YourDbContext _context;  //  Direct DbContext usage

    public async Task<CustomerDto> GetByIdAsync(Guid id)
    {
        //  Application layer knows about database queries
        var customer = await _context.Customers
```

```
            .Include(c => c.Orders)
            .FirstOrDefaultAsync(c => c.CustomerId == id);

        return _mapper.Map<CustomerDto>(customer);
    }
}
```

CORRECT:

```
// Application/Services/CustomerService.cs
public class CustomerService
{
    private readonly IUnitOfWork _unitOfWork;  //  Use abstraction

    public async Task<CustomerDto> GetByIdAsync(Guid id)
    {
        //  Application uses repository interface
        var customer = await _unitOfWork.Customers.GetByIdAsync(id);
        return _mapper.Map<CustomerDto>(customer);
    }
}

// Domain/Interfaces/ICustomerRepository.cs
public interface ICustomerRepository
{
    Task<Customer> GetByIdAsync(Guid id);
}

// Infrastructure/Repositories/CustomerRepository.cs
public class CustomerRepository : ICustomerRepository
{
    private readonly YourDbContext _context;

    public async Task<Customer> GetByIdAsync(Guid id)
    {
        //  Infrastructure handles database queries
        return await _context.Customers
            .Include(c => c.Orders)
            .FirstOrDefaultAsync(c => c.CustomerId == id);
    }
}
```

Mistake 4: Returning Entities from Controllers

WRONG:

```
// Api/Controllers/CustomersController.cs
[HttpGet("{id:guid}")]
public async Task<IActionResult> Get(Guid id)
{
    var customer = await _customerService.GetByIdAsync(id);
    return Ok(customer);  //  Exposing internal structure
}

// Application/Services/CustomerService.cs
public async Task<Customer> GetByIdAsync(Guid id)  //  Returning entity
{
    return await _unitOfWork.Customers.GetByIdAsync(id);
}
```

Problems: - Internal structure exposed to API consumers - Cannot change entity without breaking API - Security risk (might expose sensitive fields) - Circular reference issues with navigation properties

CORRECT:

```
// Api/Controllers/CustomersController.cs
[HttpGet("{id:guid}")]
public async Task<IActionResult> Get(Guid id)
{
    var customer = await _customerService.GetByIdAsync(id);
    return Ok(customer);  //  Returning DTO
}

// Application/Services/CustomerService.cs
public async Task<CustomerDto> GetByIdAsync(Guid id)  //  Returning DTO
{
    var customer = await _unitOfWork.Customers.GetByIdAsync(id);
    return _mapper.Map<CustomerDto>(customer);  //  Map to DTO
}

// Application/DTOs/CustomerDto.cs
public class CustomerDto  //  API contract, not internal structure
{
    public Guid CustomerId { get; set; }
    public string Name { get; set; }
    public string Email { get; set; }
    // Only properties that API consumers should see
}
```

Mistake 5: Manual Audit Trail

WRONG:

```
// Application/Services/OrderService.cs
public async Task<OrderDto> CreateAsync(CreateOrderRequest request)
{
    var order = _mapper.Map<Order>(request);
    order.OrderId = Guid.NewGuid();

    //  Manual audit in every service method
    order.CreatedOn = DateTime.UtcNow;
    order.CreatedBy = GetCurrentUserId();
    order.ModifiedOn = DateTime.UtcNow;
    order.ModifiedBy = GetCurrentUserId();

    await _unitOfWork.Orders.AddAsync(order);
    await _unitOfWork.SaveChangesAsync();

    return _mapper.Map<OrderDto>(order);
}
```

Problems: - Duplicated code - Easy to forget - Inconsistent across methods

CORRECT:

```
// Application/Services/OrderService.cs
public async Task<OrderDto> CreateAsync(CreateOrderRequest request)
{
    var order = _mapper.Map<Order>(request);
    order.OrderId = Guid.NewGuid();
    //  NO audit code - handled automatically

    await _unitOfWork.Orders.AddAsync(order);
    await _unitOfWork.SaveChangesAsync();  //  Triggers automatic audit

    return _mapper.Map<OrderDto>(order);
}

// Infrastructure/Data/YourDbContext.cs
public override async Task<int> SaveChangesAsync(CancellationToken cancellationToken = defau
{
    //  Automatic audit trail for ALL entities
    var userId = _currentUserService.GetUserId();
    var now = DateTime.UtcNow;

    foreach (var entry in ChangeTracker.Entries<IAuditable>())
    {
        switch (entry.State)
        {
            case EntityState.Added:
```

```
                entry.Entity.CreatedOn = now;
                entry.Entity.CreatedBy = userId;
                entry.Entity.ModifiedOn = now;
                entry.Entity.ModifiedBy = userId;
                break;
            case EntityState.Modified:
                entry.Entity.ModifiedOn = now;
                entry.Entity.ModifiedBy = userId;
                entry.Property(e => e.CreatedOn).IsModified = false;
                entry.Property(e => e.CreatedBy).IsModified = false;
                break;
        }
    }

    return await base.SaveChangesAsync(cancellationToken);
}
```

---

9. Consequences of Not Following Clean Architecture

Short-Term Consequences (Weeks to Months)

Mixed Concerns Without Clean Architecture:

```
// OrdersController.cs - EVERYTHING in one place
public class OrdersController : ControllerBase
{
    private readonly AppDbContext _context;  // Database

    [HttpPost]
    public async Task<IActionResult> CreateOrder([FromBody] Order order)  // Entity as input
    {
        // Validation
        if (string.IsNullOrEmpty(order.CustomerName))
            return BadRequest("Name required");

        // Business logic
        order.Total = order.Items.Sum(i => i.Price * i.Quantity);
        order.TaxAmount = order.Total * 0.1m;

        // Database
        _context.Orders.Add(order);
        await _context.SaveChangesAsync();

        // Email
        var client = new SmtpClient();
```

```
        await client.SendMailAsync("order@example.com", "New Order", "...");

        return Ok(order);
    }
}
```

Problems: - Cannot test business logic without database - Cannot change validation without changing controller - Cannot reuse order creation logic elsewhere - Cannot swap email provider without changing controller - 200+ line controller methods

---

Medium-Term Consequences (Months to 1 Year)

Tight Coupling **Scenario:** Need to change from SQL Server to MongoDB

Without Clean Architecture:

```
1. Find all SQL queries across entire codebase (500+ locations)
2. Rewrite each one for MongoDB
3. Change 50+ controller methods
4. Update 100+ service methods
5. Hope nothing breaks
Estimated time: 3-6 months
```

With Clean Architecture:

```
1. Create MongoCustomerRepository : ICustomerRepository
2. Create MongoOrderRepository : IOrderRepository
3. Update DI registration in Program.cs
4. Run tests
Estimated time: 1-2 weeks
```

Cannot Test **Without Clean Architecture:**

```
// OrderService.cs
public class OrderService
{
    private readonly AppDbContext _context;  // Needs real database
    private readonly SmtpClient _smtp;        // Needs real email server

    public async Task CreateOrder(Order order)
    {
        _context.Orders.Add(order);
        await _context.SaveChangesAsync();  // Requires database
        await _smtp.SendMailAsync(...);      // Sends real email
    }
}
```

```
// Testing = NIGHTMARE
[Fact]
public async Task CreateOrder_ShouldWork()
{
    // Need: Database, email server, network, etc.
    var service = new OrderService(realDatabase, realSmtp);
    // This test is SLOW and FRAGILE
}
```

With Clean Architecture:

```
// OrderService.cs
public class OrderService
{
    private readonly IUnitOfWork _unitOfWork;   // Interface
    private readonly IEmailService _emailService;  // Interface

    public async Task CreateOrder(CreateOrderRequest request)
    {
        var order = _mapper.Map<Order>(request);
        await _unitOfWork.Orders.AddAsync(order);
        await _unitOfWork.SaveChangesAsync();
        await _emailService.SendAsync(...);
    }
}

// Testing = EASY
[Fact]
public async Task CreateOrder_ShouldWork()
{
    // Mock dependencies - NO database, NO email server
    var mockUnitOfWork = new Mock<IUnitOfWork>();
    var mockEmailService = new Mock<IEmailService>();
    var service = new OrderService(mockUnitOfWork.Object, mockEmailService.Object);

    // Fast, reliable, isolated test
    await service.CreateOrder(request);

    mockUnitOfWork.Verify(x => x.SaveChangesAsync(), Times.Once);
}
```

Long-Term Consequences (1-3 Years)

"Big Ball of Mud" Architecture Without Clean Architecture after 2 years:

```
src/
  Controllers/
      OrdersController.cs (2,500 lines)  ← Everything here
      CustomersController.cs (1,800 lines)
      ProductsController.cs (3,200 lines)
  Models/
      Order.cs (mixed entity/DTO/validation)
      Customer.cs (mixed entity/DTO/validation)
  Data/
       AppDbContext.cs (5,000 lines)  ← All SQL here
```

Problems: - Cannot find anything - Code reviews take days - New features break existing features - Cannot onboard new developers (takes months) - Small changes require massive testing - Technical debt is crushing - **Developers quit**

With Clean Architecture after 2 years:

```
src/
  Domain/
      Entities/ (50 clean entity files)
      Interfaces/ (20 interface files)
  Application/
      Services/ (30 focused service files, ~200 lines each)
      DTOs/ (60 DTO files)
      Validators/ (30 validator files)
  Infrastructure/
      Repositories/ (20 repository files, ~100 lines each)
      Data/ (1 DbContext, clean configurations)
  Api/
       Controllers/ (30 thin controllers, ~50 lines each)
```

Benefits: - Easy to find anything - Code reviews take minutes - New features don't break old features - New developers productive in days - Changes are isolated and safe - Low technical debt - **Developers love working here**

Cannot Scale Team Without Clean Architecture: - 5 developers → constant merge conflicts - 10 developers → impossible to coordinate - 20 developers → total chaos

With Clean Architecture: - 5 developers → work independently on different layers - 10 developers → team A works on features, team B works on infrastructure - 20 developers → multiple teams, minimal conflicts

---

Business Impact

Metric	Without Clean Architecture	With Clean Architecture
Time to add feature	2-4 weeks	2-4 days
Bug fix time	1-3 days	1-3 hours
Test coverage	20-30%	80-95%
Onboarding time	3-6 months	1-2 weeks
Technical debt	High, growing	Low, controlled
Developer turnover	High	Low
Production incidents	Frequent	Rare
Deployment confidence	Low	High

---

10. Step-by-Step Development Workflow

Workflow for Adding a New Feature

Example: Add "Product Reviews" feature

Step 1: Define Domain (Business Requirements) **Think:** What is a ProductReview in our business?

```
// Domain/Entities/ProductReview.cs
namespace YourProject.Domain.Entities;

public class ProductReview : IAuditable
{
    public Guid ReviewId { get; set; }
    public Guid ProductId { get; set; }
    public Guid CustomerId { get; set; }
    public int Rating { get; set; }  // 1-5
    public string Comment { get; set; }
    public bool IsVerifiedPurchase { get; set; }

    // Audit properties
    public DateTime CreatedOn { get; set; }
    public Guid? CreatedBy { get; set; }
    public DateTime ModifiedOn { get; set; }
    public Guid? ModifiedBy { get; set; }

    // Navigation properties
    public virtual Product Product { get; set; }
    public virtual Customer Customer { get; set; }
}
```

```
// Domain/Interfaces/IProductReviewRepository.cs
public interface IProductReviewRepository
{
    Task<ProductReview> GetByIdAsync(Guid id, CancellationToken cancellationToken = default)
    Task<IEnumerable<ProductReview>> GetByProductIdAsync(Guid productId, CancellationToken c
    Task AddAsync(ProductReview review, CancellationToken cancellationToken = default);
    void Delete(ProductReview review);
}
```

Step 2: Define Application Layer (Use Cases) **Think:** What operations can users perform?

```
// Application/DTOs/CreateProductReviewRequest.cs
public class CreateProductReviewRequest
{
    public Guid ProductId { get; set; }
    public int Rating { get; set; }
    public string Comment { get; set; }
}

// Application/DTOs/ProductReviewDto.cs
public class ProductReviewDto
{
    public Guid ReviewId { get; set; }
    public Guid ProductId { get; set; }
    public string CustomerName { get; set; }
    public int Rating { get; set; }
    public string Comment { get; set; }
    public bool IsVerifiedPurchase { get; set; }
    public DateTime CreatedOn { get; set; }
}

// Application/Validators/CreateProductReviewRequestValidator.cs
public class CreateProductReviewRequestValidator : AbstractValidator<CreateProductReviewRequ
{
    public CreateProductReviewRequestValidator()
    {
        RuleFor(x => x.ProductId)
            .NotEmpty().WithMessage("Product ID is required");

        RuleFor(x => x.Rating)
            .InclusiveBetween(1, 5).WithMessage("Rating must be between 1 and 5");

        RuleFor(x => x.Comment)
            .NotEmpty().WithMessage("Comment is required")
            .MaximumLength(1000).WithMessage("Comment cannot exceed 1000 characters");
```

```
    }
}

// Application/Interfaces/IProductReviewService.cs
public interface IProductReviewService
{
    Task<ProductReviewDto> CreateAsync(CreateProductReviewRequest request, CancellationToken
    Task<IEnumerable<ProductReviewDto>> GetByProductIdAsync(Guid productId, CancellationToke
    Task DeleteAsync(Guid id, CancellationToken cancellationToken = default);
}

// Application/Services/ProductReviewService.cs
public class ProductReviewService : IProductReviewService
{
    private readonly IUnitOfWork _unitOfWork;
    private readonly IMapper _mapper;
    private readonly ICurrentUserService _currentUserService;

    public ProductReviewService(
        IUnitOfWork unitOfWork,
        IMapper mapper,
        ICurrentUserService currentUserService)
    {
        _unitOfWork = unitOfWork;
        _mapper = mapper;
        _currentUserService = currentUserService;
    }

    public async Task<ProductReviewDto> CreateAsync(
        CreateProductReviewRequest request,
        CancellationToken cancellationToken = default)
    {
        // Map DTO to entity
        var review = _mapper.Map<ProductReview>(request);
        review.ReviewId = Guid.NewGuid();
        review.CustomerId = _currentUserService.GetUserId();

        // Check if verified purchase (business logic)
        var hasPurchased = await _unitOfWork.Orders
            .HasCustomerPurchasedProductAsync(review.CustomerId, review.ProductId, cancellat
        review.IsVerifiedPurchase = hasPurchased;

        // Save (audit trail handled automatically)
        await _unitOfWork.ProductReviews.AddAsync(review, cancellationToken);
        await _unitOfWork.SaveChangesAsync(cancellationToken);
```

```
        return _mapper.Map<ProductReviewDto>(review);
    }

    public async Task<IEnumerable<ProductReviewDto>> GetByProductIdAsync(
        Guid productId,
        CancellationToken cancellationToken = default)
    {
        var reviews = await _unitOfWork.ProductReviews.GetByProductIdAsync(productId, cancel
        return _mapper.Map<IEnumerable<ProductReviewDto>>(reviews);
    }

    public async Task DeleteAsync(Guid id, CancellationToken cancellationToken = default)
    {
        var review = await _unitOfWork.ProductReviews.GetByIdAsync(id, cancellationToken);
        if (review == null)
            throw new KeyNotFoundException($"Review with ID {id} not found");

        _unitOfWork.ProductReviews.Delete(review);
        await _unitOfWork.SaveChangesAsync(cancellationToken);
    }
}

// Application/Mappings/ProductReviewMappingProfile.cs
public class ProductReviewMappingProfile : Profile
{
    public ProductReviewMappingProfile()
    {
        CreateMap<CreateProductReviewRequest, ProductReview>();

        CreateMap<ProductReview, ProductReviewDto>()
            .ForMember(dest => dest.CustomerName,
                       opt => opt.MapFrom(src => src.Customer.Name));
    }
}
```

Step 3: Implement Infrastructure (Data Access) **Think:** How do we store and retrieve reviews?

```
// Infrastructure/Repositories/ProductReviewRepository.cs
public class ProductReviewRepository : IProductReviewRepository
{
    private readonly YourDbContext _context;

    public ProductReviewRepository(YourDbContext context)
    {
        _context = context;
```

```
    }

    public async Task<ProductReview> GetByIdAsync(Guid id, CancellationToken cancellationTok
    {
        return await _context.ProductReviews
            .Include(r => r.Customer)
            .Include(r => r.Product)
            .FirstOrDefaultAsync(r => r.ReviewId == id, cancellationToken);
    }

    public async Task<IEnumerable<ProductReview>> GetByProductIdAsync(
        Guid productId,
        CancellationToken cancellationToken = default)
    {
        return await _context.ProductReviews
            .Include(r => r.Customer)
            .Where(r => r.ProductId == productId)
            .OrderByDescending(r => r.CreatedOn)
            .ToListAsync(cancellationToken);
    }

    public async Task AddAsync(ProductReview review, CancellationToken cancellationToken = c
    {
        await _context.ProductReviews.AddAsync(review, cancellationToken);
    }

    public void Delete(ProductReview review)
    {
        _context.ProductReviews.Remove(review);
    }
}

// Infrastructure/Data/YourDbContext.cs (add configuration)
public DbSet<ProductReview> ProductReviews { get; set; }

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<ProductReview>(entity =>
    {
        entity.ToTable("product_reviews", "dbo");
        entity.HasKey(e => e.ReviewId);

        entity.Property(e => e.ReviewId).HasColumnName("review_id");
        entity.Property(e => e.ProductId).HasColumnName("product_id");
        entity.Property(e => e.CustomerId).HasColumnName("customer_id");
        entity.Property(e => e.Rating).HasColumnName("rating");
```

```
        entity.Property(e => e.Comment).HasColumnName("comment").HasMaxLength(1000);
        entity.Property(e => e.IsVerifiedPurchase).HasColumnName("is_verified_purchase");
        entity.Property(e => e.CreatedOn).HasColumnName("created_on");
        entity.Property(e => e.CreatedBy).HasColumnName("created_by");
        entity.Property(e => e.ModifiedOn).HasColumnName("modified_on");
        entity.Property(e => e.ModifiedBy).HasColumnName("modified_by");

        entity.HasOne(e => e.Product)
              .WithMany()
              .HasForeignKey(e => e.ProductId);

        entity.HasOne(e => e.Customer)
              .WithMany()
              .HasForeignKey(e => e.CustomerId);
    });
}
```

Step 4: Create API Endpoints **Think:** How do external clients interact with reviews?

```
// Api/Controllers/ProductReviewsController.cs
[ApiController]
[Route("api/products/{productId:guid}/reviews")]
[Produces("application/json")]
public class ProductReviewsController : ControllerBase
{
    private readonly IProductReviewService _productReviewService;
    private readonly ILogger<ProductReviewsController> _logger;

    public ProductReviewsController(
        IProductReviewService productReviewService,
        ILogger<ProductReviewsController> logger)
    {
        _productReviewService = productReviewService;
        _logger = logger;
    }

    /// <summary>
    /// Get all reviews for a product
    /// </summary>
    [HttpGet]
    [ProducesResponseType(typeof(IEnumerable<ProductReviewDto>), StatusCodes.Status200OK)]
    public async Task<IActionResult> GetByProduct(Guid productId, CancellationToken cancella
    {
        try
        {
```

```
            var reviews = await _productReviewService.GetByProductIdAsync(productId, cancell
            return Ok(reviews);
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, "Error retrieving reviews for product {ProductId}", product
            return StatusCode(500, "An error occurred while retrieving reviews");
        }
    }

    /// <summary>
    /// Create a new product review
    /// </summary>
    [HttpPost]
    [ProducesResponseType(typeof(ProductReviewDto), StatusCodes.Status201Created)]
    [ProducesResponseType(StatusCodes.Status400BadRequest)]
    public async Task<IActionResult> Post(
        Guid productId,
        [FromBody] CreateProductReviewRequest request,
        CancellationToken cancellationToken)
    {
        try
        {
            if (!ModelState.IsValid)
                return BadRequest(ModelState);

            // Ensure product ID matches route
            request.ProductId = productId;

            var review = await _productReviewService.CreateAsync(request, cancellationToken)
            return CreatedAtAction(nameof(Get), new { id = review.ReviewId }, review);
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, "Error creating review for product {ProductId}", productId)
            return StatusCode(500, "An error occurred while creating the review");
        }
    }

    /// <summary>
    /// Delete a product review
    /// </summary>
    [HttpDelete("{id:guid}")]
    [ProducesResponseType(StatusCodes.Status204NoContent)]
    [ProducesResponseType(StatusCodes.Status404NotFound)]
    public async Task<IActionResult> Delete(Guid id, CancellationToken cancellationToken)
```

```
    {
        try
        {
            await _productReviewService.DeleteAsync(id, cancellationToken);
            return NoContent();
        }
        catch (KeyNotFoundException)
        {
            return NotFound($"Review with ID {id} not found");
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, "Error deleting review {ReviewId}", id);
            return StatusCode(500, "An error occurred while deleting the review");
        }
    }
}

// Api/Program.cs (register dependencies)
builder.Services.AddScoped<IProductReviewRepository, ProductReviewRepository>();
builder.Services.AddScoped<IProductReviewService, ProductReviewService>();
```

Step 5: Write Tests

```
// tests/YourProject.UnitTests/Services/ProductReviewServiceTests.cs
public class ProductReviewServiceTests
{
    private readonly Mock<IUnitOfWork> _mockUnitOfWork;
    private readonly Mock<IMapper> _mockMapper;
    private readonly Mock<ICurrentUserService> _mockCurrentUserService;
    private readonly ProductReviewService _service;

    public ProductReviewServiceTests()
    {
        _mockUnitOfWork = new Mock<IUnitOfWork>();
        _mockMapper = new Mock<IMapper>();
        _mockCurrentUserService = new Mock<ICurrentUserService>();

        _service = new ProductReviewService(
            _mockUnitOfWork.Object,
            _mockMapper.Object,
            _mockCurrentUserService.Object);
    }

    [Fact]
    public async Task CreateAsync_ShouldCreateReview_WithVerifiedPurchase()
```

```
    {
        // Arrange
        var request = new CreateProductReviewRequest
        {
            ProductId = Guid.NewGuid(),
            Rating = 5,
            Comment = "Great product!"
        };

        var userId = Guid.NewGuid();
        _mockCurrentUserService.Setup(x => x.GetUserId()).Returns(userId);

        var review = new ProductReview
        {
            ProductId = request.ProductId,
            Rating = request.Rating,
            Comment = request.Comment,
            CustomerId = userId
        };

        _mockMapper.Setup(x => x.Map<ProductReview>(request)).Returns(review);

        // Customer has purchased the product
        _mockUnitOfWork.Setup(x => x.Orders.HasCustomerPurchasedProductAsync(userId, request
                       .ReturnsAsync(true);

        // Act
        var result = await _service.CreateAsync(request);

        // Assert
        _mockUnitOfWork.Verify(x => x.ProductReviews.AddAsync(It.IsAny<ProductReview>(), def
        _mockUnitOfWork.Verify(x => x.SaveChangesAsync(default), Times.Once);
        review.IsVerifiedPurchase.Should().BeTrue();
    }
}
```

Step 6: Create Migration and Update Database

```
# Create migration
cd src/YourProject.Infrastructure
dotnet ef migrations add AddProductReviews --startup-project ../YourProject.Api

# Apply migration
dotnet ef database update --startup-project ../YourProject.Api
```

Step 7: Test Manually

```
# Run application
dotnet run --project src/YourProject.Api/YourProject.Api.csproj

# Test endpoint
curl -X POST http://localhost:5000/api/products/123e4567-e89b-12d3-a456-426614174000/reviews
  -H "Content-Type: application/json" \
  -d '{
    "rating": 5,
    "comment": "Excellent product, highly recommend!"
  }'
```

---

11. Real Examples from OfferCraft.Lobby

Example 1: Lobby Creation Flow

User Request: `POST /api/lobbies` with `CreateLobbyRequest`

1. API Layer (Entry Point)

```
// Api/Controllers/LobbiesController.cs
[HttpPost]
public async Task<IActionResult> Post([FromBody] CreateLobbyRequest request, CancellationTok
{
    // Step 1: Validate (FluentValidation runs automatically)
    if (!ModelState.IsValid)
        return BadRequest(ModelState);

    // Step 2: Delegate to service
    var lobby = await _lobbyService.CreateAsync(request, cancellationToken);

    // Step 3: Return HTTP 201 with location
    return CreatedAtAction(nameof(Get), new { id = lobby.LobbyId }, lobby);
}
```

2. Application Layer (Business Logic)

```
// Application/Services/LobbyService.cs
public async Task<LobbyDto> CreateAsync(CreateLobbyRequest request, CancellationToken cancel
{
    // Step 1: Map DTO to Entity
    var lobby = _mapper.Map<Lobby>(request);

    // Step 2: Set business defaults
    lobby.LobbyId = Guid.NewGuid();
    lobby.IsArchived = false;
    lobby.IsDisabled = false;
```

```
    // Step 3: Save (NO manual audit - handled automatically by Infrastructure)
    await _unitOfWork.Lobbies.AddAsync(lobby, cancellationToken);
    await _unitOfWork.SaveChangesAsync(cancellationToken);

    // Step 4: Map Entity to DTO for response
    return _mapper.Map<LobbyDto>(lobby);
}
```

3. Infrastructure Layer (Automatic Audit)

```
// Infrastructure/Data/LobbyDbContext.cs
public override async Task<int> SaveChangesAsync(CancellationToken cancellationToken = defau
{
    var userId = _currentUserService.GetUserId();  // Get from HTTP context
    var now = DateTime.UtcNow;

    foreach (var entry in ChangeTracker.Entries<IAuditable>())
    {
        if (entry.State == EntityState.Added)
        {
            // Automatically set audit fields
            entry.Entity.CreatedOn = now;
            entry.Entity.CreatedBy = userId;
            entry.Entity.ModifiedOn = now;
            entry.Entity.ModifiedBy = userId;
        }
    }

    return await base.SaveChangesAsync(cancellationToken);
}
```

4. Domain Layer (Business Entity)

```
// Domain/Entities/Lobby.cs
public class Lobby : IAuditable, ISoftDeletable
{
    public Guid LobbyId { get; set; }
    public string Title { get; set; }
    // ... business properties

    // Audit properties (set automatically by Infrastructure)
    public DateTime CreatedOn { get; set; }
    public Guid? CreatedBy { get; set; }
    public DateTime ModifiedOn { get; set; }
    public Guid? ModifiedBy { get; set; }
}
```

Key Observations

Separation of Concerns: - Controller handles HTTP - Service handles business logic - DbContext handles audit trail - Entity is pure business model

Dependency Flow: - API → Application → Infrastructure → Domain - Domain knows nothing about outer layers

Testability: - Can test service without controller - Can test service without database - Can test business logic in isolation

---

Example 2: Lobby Update with Audit Preservation

User Request: `PUT /api/lobbies/{id}` with `UpdateLobbyRequest`

Database BEFORE:

```
lobby_id: 66509f36-24f1-4cc9-a80b-f52b647d89f3
created_on: 2026-01-19 14:47:53.868350
created_by: 00000000-0000-0000-0000-000000000001
modified_on: 2026-01-19 14:47:53.868350  ← Same as created
modified_by: 00000000-0000-0000-0000-000000000001
```

1. Service Layer (Business Logic)

```
// Application/Services/LobbyService.cs
public async Task<LobbyDto> UpdateAsync(Guid id, UpdateLobbyRequest request, CancellationTok
{
    // Get existing entity
    var lobby = await _unitOfWork.Lobbies.GetByIdAsync(id, cancellationToken);
    if (lobby == null)
        throw new KeyNotFoundException($"Lobby with ID {id} not found");

    // Update only business properties
    lobby.Title = request.Title;
    lobby.Description = request.Description;
    lobby.IdleTimeout = request.IdleTimeout;
    // NO audit code here!

    // Save (audit handled automatically)
    _unitOfWork.Lobbies.Update(lobby);
    await _unitOfWork.SaveChangesAsync(cancellationToken);

    return _mapper.Map<LobbyDto>(lobby);
}
```

2. Infrastructure Layer (Audit Preservation)

```csharp
// Infrastructure/Data/LobbyDbContext.cs
public override async Task<int> SaveChangesAsync(CancellationToken cancellationToken = defau
{
    var userId = _currentUserService.GetUserId();
    var now = DateTime.UtcNow;

    foreach (var entry in ChangeTracker.Entries<IAuditable>())
    {
        if (entry.State == EntityState.Modified)
        {
            // Update only modified fields
            entry.Entity.ModifiedOn = now;
            entry.Entity.ModifiedBy = userId;

            // CRITICAL: Prevent overwriting created fields
            entry.Property(e => e.CreatedOn).IsModified = false;
            entry.Property(e => e.CreatedBy).IsModified = false;
        }
    }

    return await base.SaveChangesAsync(cancellationToken);
}
```

Database AFTER:

```
lobby_id: 66509f36-24f1-4cc9-a80b-f52b647d89f3
created_on: 2026-01-19 14:47:53.868350  ← PRESERVED!
created_by: 00000000-0000-0000-0000-000000000001  ← PRESERVED!
modified_on: 2026-01-19 14:52:07.825858  ← UPDATED!
modified_by: 00000000-0000-0000-0000-000000000001  ← UPDATED!
```

Key Observations

Automatic Audit: - Service has ZERO audit code - Infrastructure handles all audit logic - Created fields preserved automatically - Modified fields updated automatically

Consistency: - ALL entities get same audit treatment - Impossible to forget audit fields - Consistent across entire application

---

12. Testing Strategy

Test Pyramid

```
      /\
     /  \      E2E Tests (Few)
```

```
      /____\     - Full API tests
     /      \    - Database integration
    /        \
   /----------\  Integration Tests (Some)
  /            \ - Service + Repository
 /______________\ - In-memory database
/                \
|________________| Unit Tests (Many)
                   - Service logic
                   - Validators
                   - Mappers
```

Unit Tests (Fast, Isolated)

What to Test: - Business logic in services - Validation rules - Mapping configurations - Domain entity behavior

Example:

```
// tests/YourProject.UnitTests/Services/LobbyServiceTests.cs
public class LobbyServiceTests
{
    private readonly Mock<IUnitOfWork> _mockUnitOfWork;
    private readonly Mock<IMapper> _mockMapper;
    private readonly LobbyService _service;

    public LobbyServiceTests()
    {
        _mockUnitOfWork = new Mock<IUnitOfWork>();
        _mockMapper = new Mock<IMapper>();
        _service = new LobbyService(_mockUnitOfWork.Object, _mockMapper.Object);
    }

    [Fact]
    public async Task CreateAsync_ShouldCreateLobby_WithCorrectDefaults()
    {
        // Arrange
        var request = new CreateLobbyRequest
        {
            Title = "Test Lobby",
            AccountId = Guid.NewGuid()
        };

        var lobby = new Lobby { Title = request.Title };
        _mockMapper.Setup(x => x.Map<Lobby>(request)).Returns(lobby);

        // Act
```

```
        await _service.CreateAsync(request);

        // Assert
        lobby.IsArchived.Should().BeFalse();
        lobby.IsDisabled.Should().BeFalse();
        lobby.LobbyId.Should().NotBeEmpty();
        _mockUnitOfWork.Verify(x => x.SaveChangesAsync(default), Times.Once);
    }
}
```

Integration Tests (Moderate Speed, Real Components)

What to Test: - Full API endpoints - Repository + Database interactions - Automatic audit trail - Transaction handling

Example:

```
// tests/YourProject.IntegrationTests/Controllers/LobbiesControllerTests.cs
public class LobbiesControllerTests : IClassFixture<WebApplicationFactory<Program>>
{
    private readonly HttpClient _client;

    public LobbiesControllerTests(WebApplicationFactory<Program> factory)
    {
        _client = factory.WithWebHostBuilder(builder =>
        {
            builder.ConfigureServices(services =>
            {
                // Use in-memory database
                services.AddDbContext<LobbyDbContext>(options =>
                    options.UseInMemoryDatabase("TestDb"));
            });
        }).CreateClient();
    }

    [Fact]
    public async Task POST_CreateLobby_ReturnsCreatedLobby()
    {
        // Arrange
        var request = new CreateLobbyRequest
        {
            Title = "Integration Test Lobby",
            AccountId = Guid.NewGuid(),
            LobbyTypeId = 1,
            LobbyModeId = 1,
            RequiresLogin = true,
            LobbyCampaignOrderTypeId = 1
```

```
        };

        // Act
        var response = await _client.PostAsJsonAsync("/api/lobbies", request);

        // Assert
        response.StatusCode.Should().Be(HttpStatusCode.Created);
        var lobby = await response.Content.ReadFromJsonAsync<LobbyDto>();
        lobby.Title.Should().Be(request.Title);
        lobby.LobbyId.Should().NotBeEmpty();
        lobby.CreatedOn.Should().BeCloseTo(DateTime.UtcNow, TimeSpan.FromSeconds(5));
    }
}
```

---

13. Best Practices Checklist

Before Starting Development

- ☐ **Understand the business domain** - What problems are we solving?
- ☐ **Design domain entities first** - What are the core business concepts?
- ☐ **Define interfaces** - What operations do we need?
- ☐ **Plan DTOs** - What data flows in/out of the API?

During Development

Domain Layer

- ☐ Zero external dependencies (no NuGet packages except domain libraries)
- ☐ No database attributes (`[Table]`, `[Column]`, etc.)
- ☐ No framework dependencies (no EF Core, ASP.NET, etc.)
- ☐ Pure business entities with business properties only
- ☐ Interfaces define contracts, not implementations

Application Layer

- ☐ Only depends on Domain
- ☐ NO database code (no `DbContext`, no SQL)
- ☐ Services use repository interfaces, not implementations
- ☐ All DTOs defined here (never expose entities to API)
- ☐ FluentValidation for all input validation
- ☐ AutoMapper for entity DTO mapping
- ☐ NO audit code (handled by Infrastructure)

Infrastructure Layer

- ☐ Implements Domain interfaces

- ☐ All EF Core code here (DbContext, configurations, migrations)
- ☐ Repository implementations here
- ☐ External service integrations here
- ☐ Automatic audit trail in `SaveChangesAsync`
- ☐ Soft delete handling in `SaveChangesAsync`

API Layer

- ☐ Controllers are THIN (< 100 lines)
- ☐ No business logic in controllers
- ☐ No database access in controllers
- ☐ Returns DTOs, never entities
- ☐ Proper HTTP status codes (200, 201, 400, 404, 500)
- ☐ Comprehensive error handling
- ☐ Logging for all operations

Code Review Checklist

- ☐ **Dependency direction correct?** (inward only)
- ☐ **Domain has zero dependencies?** (check .csproj)
- ☐ **Services use abstractions?** (interfaces, not concrete classes)
- ☐ **Controllers delegate to services?** (no business logic)
- ☐ **DTOs used for API?** (not entities)
- ☐ **Validation in place?** (FluentValidation)
- ☐ **Tests written?** (unit + integration)
- ☐ **Audit trail automatic?** (no manual code)
- ☐ **Error handling present?** (try/catch, logging)
- ☐ **Documentation updated?** (XML comments, README)

Common Red Flags

Domain Layer: - `using Microsoft.EntityFrameworkCore;` → WRONG! - `[Table("lobbies")]` on entity → WRONG! - Entity with `DbContext` dependency → WRONG!

Application Layer: - `using Microsoft.AspNetCore.Mvc;` → WRONG! - Service with `DbContext` parameter → WRONG! - Returning `Lobby` instead of `LobbyDto` → WRONG!

Controllers: - Business logic in controller → WRONG! - Manual audit field setting → WRONG! - Direct `DbContext` usage → WRONG!

Summary

The Golden Rules

1. **Domain is king** - Pure business logic, zero dependencies

2. **Dependencies point inward** - Outer layers depend on inner layers
3. **Use interfaces** - Depend on abstractions, not implementations
4. **DTOs for API** - Never expose entities
5. **Thin controllers** - Delegate to services
6. **Automatic cross-cutting concerns** - Audit, logging in Infrastructure
7. **Test everything** - Unit tests for logic, integration tests for flow

Quick Decision Guide

"Where does this code go?" - Is it a business concept? → **Domain** - Does it orchestrate multiple entities? → **Application** - Does it access a database or external API? → **Infrastructure** - Does it handle HTTP? → **API**

The Benefits You Get

Testability - Fast, isolated unit tests
Maintainability - Easy to find and change code
Scalability - Team can work in parallel
Flexibility - Swap databases/frameworks easily
Documentation - Structure IS documentation
Onboarding - New developers productive quickly
Quality - Fewer bugs, better code reviews
Longevity - Code lasts years without rot

Additional Resources

Official Documentation

- Microsoft Clean Architecture
- Entity Framework Core
- ASP.NET Core

Books

- **Clean Architecture** by Robert C. Martin
- **Domain-Driven Design** by Eric Evans
- **Implementing Domain-Driven Design** by Vaughn Vernon

Sample Projects

- **OfferCraft.Lobby.Microservice** (this repository)
- Microsoft eShopOnContainers
- Clean Architecture Template by Jason Taylor

Document Version: 1.0
Last Updated: January 27, 2026
Maintained By: Architecture Team
Questions? Contact: architecture@offercraft.com

Appendix: FAQs

Q: Can I put validators in Domain layer?

A: NO. Validators validate INPUT from API (DTOs), not business entities. They belong in Application layer.

Q: Can Application layer reference Infrastructure?

A: NO. This violates dependency rules. Application depends only on Domain. Use dependency injection to inject Infrastructure implementations.

Q: Where do I put AutoMapper profiles?

A: Application layer. They map between DTOs (Application) and Entities (Domain).

Q: Can I use EF Core attributes on entities?

A: NO. Use Fluent API in Infrastructure/Data/DbContext instead. Keep entities pure.

Q: Where does caching go?

A: Infrastructure layer. Create `ICacheService` in Application, implement in Infrastructure.

Q: Can I skip a layer for simple CRUD?

A: NO. Consistency is more important than convenience. Follow the pattern always.

Q: What if I have shared code between projects?

A: Create a **Shared** project at the same level as Domain. Other projects can reference it.

END OF GUIDE

Use this guide for ALL microservice development in the organization.